NOVA IMS INFORMATION MANAGEMENT SCHOOL

List comprehension: the rule, not the steps

Computation I · Lecture 10

Karina Brotto Rebuli
krebuli@novaims.unl.pt

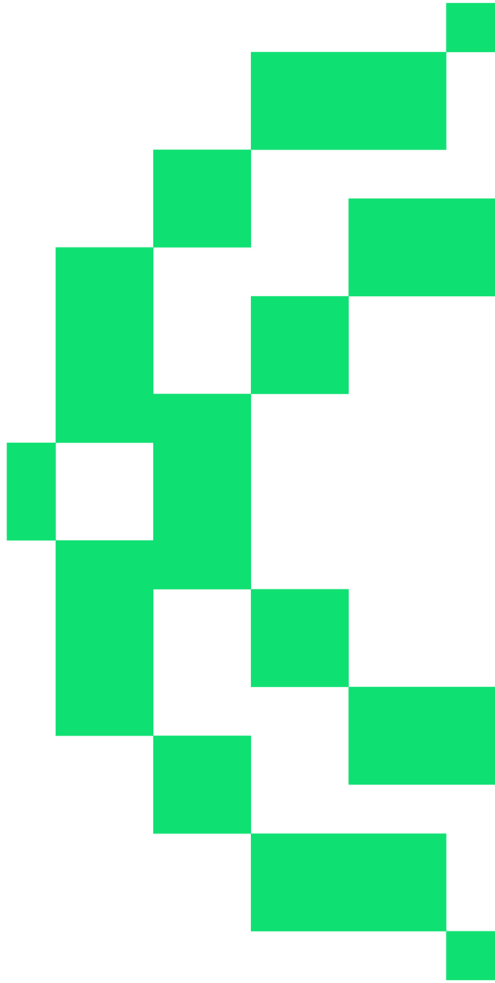
The three lines that worked

Exercise 15, replayed — a list of each name's length

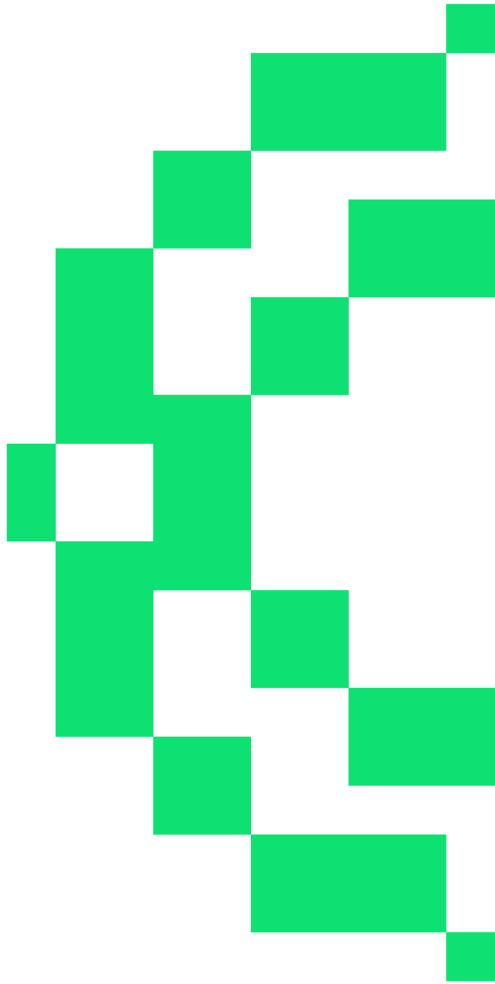
```
names = ["Ada", "Grace", "Ken"]
lengths = []
for name in names:
    lengths.append(len(name))
print(lengths)
```

Many lines for a single list

This program runs, but is long: an empty list, a loop, an append. Today the same list comes from a single line that says what it should contain.



```
"""One line for what a loop took many to say."""
```



Jack Schwartz

1969

The problem: to work with a collection, you walked it one element at a time — open an empty list, step through, do the work, append, and repeat. He wanted to state what a collection is in a single line, the way mathematics names a set by the rule its members obey.



Image source

Three questions today

Build a whole list in a single line — what goes inside the brackets?

Put two loops in one line — in what order do they run?

Can the line choose which elements enter the list?

01.

The list, in one line

Say what it contains, not how to build it

The three lines, taken apart

Minimal steps to do the work with loops

```
names = ["Ada", "Grace", "Ken"]
lengths = []
for name in names:
    lengths.append(len(name))
print(lengths)
```

```
>>> [3, 5, 3]
```

Start with an empty list
`lengths = []`

Loop through each item of a sequence
`for name in names:`

The expression defining each list item
`lengths.append(len(name))`

The three lines becoming one

[expression for item in sequence]

THE LOOP

An empty list first — then one item append at a time

Start with an empty list
`lengths = []`

Loop through each item
`for name in names:`

The expression defining each list item
`lengths.append(len(name))`

“For each item of this sequence, do this and append.”

THE LIST COMPREHENSION

No empty list to start — the brackets do it

```
lengths = [ len(name) for name in names ]
```

the expression — the rule

“Do this for each item of this sequence.”

The three parts of a comprehension

The brackets make a list, the expression defines each item, the for-clause defines the repetition rule

```
L = [0 for i in range(10)]
print(L)
```

```
>>> [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

LIVE

Write it in parts, fold it into brackets

This is legal and can help while you are learning

Step 1 — write the expression of each item on one line, and the repetition rule — the `for` — on another.
Step 2 — join everything on the line of the brackets.

```
names = ["Ada", "Grace", "Ken"]
lengths = [len(name) for name in names]
print(lengths)
```

```
>>> [3, 5, 3]
```

Your turn · Build it twice

Build the list below twice, on paper, from `range(1, 6)`.

```
>>> [2, 4, 6, 8, 10]
```

THE LOOP

THE LIST COMPREHENSION

LIVE

The sequence can be a string

`for` hands out one letter at a time — the expression turns each one into a capital

```
word = "python"
capitals = [ch.upper() for ch in word]
print(capitals)
```

```
>>> ['P', 'Y', 'T', 'H', 'O', 'N']
```

LIVE

Convert every piece at once

One comprehension turns the list of strings into a list of numbers

```
text = "3.5 4.0 2.0"
numbers = [float(e) for e in text.split()]
print(numbers)
```

```
>>> [3.5, 4.0, 2.0]
```

LIVE

Your turn · Build this list in one line

Build the list below in one line, then print it. On paper, before any run. Use this lecture's comprehension and range(); the rule for stepping and the expression are yours to find.

```
>>> [1, 9, 25, 49, 81]
```

02.

Two loops, one line

A comprehension can loop twice

The grid that worked

Exercise 16, replayed — every [row, col] pair in a 3-by-3 grid

```
pairs = []
for i in range(3):
    for j in range(3):
        pairs.append([i, j])
print(pairs)
```

```
>>> [[0, 0], [0, 1], ... [2, 2]]
```

The four lines, taken apart

Minimal steps to do the work with nested loops

```
pairs = []
for i in range(2):
    for j in range(3):
        pairs.append([i, j])
print(pairs)
```

Start with an empty list

```
pairs = []
```

Loop through each item of each sequence

```
for i in range(2):
    for j in range(3):
```

The expression defining each list item

```
pairs.append([i, j])
```

Two loops fold into one line

Two for-clauses, read left to right — in the order you would nest them

THE LOOP

An empty list first — then one item append at a time

Start with an empty list

```
pairs = []
```

Loop through each item of each sequence

```
for i in range(2):
    for j in range(3):
```

The expression defining each list item

```
pairs.append([i, j])
```

“For each item in the sequence of the outer loop combined with each item in the sequence of the inner loop, do this and append.”

THE LIST COMPREHENSION

No empty list to start — the brackets do it

```
pairs = [ [i, j] for i in range(2) for j in range(3) ]
```

the expression

the rule

“Do this for each item in sequence 1 combined with each item in sequence 2.”

LIVE

Your turn · Build it twice, with two loops

Build the list below twice, on paper, from the strings `"ab"` and `"xy"`. First with two nested `for` loops and `append`. Then fold your own loops into a comprehension with two for-clauses — same list, both ways.

```
>>> ['ax', 'ay', 'bx', 'by']
```

THE LOOP

THE LIST COMPREHENSION

This is still valid

You can indent the elements inside the brackets to see the comprehension's structure

```
combos = [c1 + c2 for c1 in "ab" for c2 in "xy"]
print(combos)
```

```
>>> ['ax', 'ay', 'bx', 'by']
```

LIVE

The expression may compute with both variables

How many values do r and c assume? What is printed? Trace table on paper before — commit after

```
my_list = [10 * r + c for r in range(3) for c in range(3)]
print(my_list)
```

```
>>> [0, 1, 2, 10, 11, 12, 20, 21, 22]
```

03.

The line can filter

An `if` at the end chooses what enters the list

LIVE

In-line conditional: filtering items

From left to right, and each element faces the test — only the ones that answer `True` enter the list

THE LOOP

```
nums = [4, 7, 2, 9, 1]
evens = []
for n in nums:
    if n % 2 == 0:
        evens.append(n)
```

THE LIST COMPREHENSION

```
nums = [4, 7, 2, 9, 1]
evens = [n for n in nums if n % 2 == 0]
```

As we have seen with the nested `for`: in in-line statements, what is indented comes to the right side — without the colons.

LIVE

The test is yours to write

Any expression that answers `True` or `False` — week 3's indexing picks the names that start with A

```
names = ["Ana", "Grace", "Alan", "Ken"]
a_names = [n for n in names if n[0] == "A"]
print(a_names)
```

```
>>> ['Ana', 'Alan']
```

LIVE

The filter works with two loops

The `if` comes last, after both `for`-clauses — the pairs that fail the test never enter

```
combos = [c1 + c2 for c1 in "ab" for c2 in "ab" if c1 != c2]
print(combos)
```

```
>>> ['ab', 'ba']
```

LIVE

Your turn · Keep the positives

Build the list below in one line from the list given in the panel, then print it. On paper, before any run. Use this lecture's filter: an `if` after the `for`-clause.

```
nums = [3, -1, 7, -5, 2]
# your comprehension here
```

```
>>> [3, 7, 2]
```

What holds when you leave

In one line each

A list in one line

[expression for item in sequence] — say what each element is, and the loop lives only inside the brackets

Two loops, one line

two for-clauses read left to right — the first is the outer loop, the last is the inner loop

The line can filter

an `if` after the `for`-clauses keeps only the elements that pass the test — the rest never enter

Before the next theoretical lecture

On your own

Think Python — Section 18.5 List comprehensions.

One habit — if a comprehension does not give you what you meant, write it out with indentation first. If it still does not work, write it back out as a loop and run that. The mistake is almost always in the expression.

Exercise 1 · Predict what it prints

Read the program below and write exactly the list it prints. On paper, before any run. Then run it and compare.

```
nums = [1, 2, 3, 4]
doubled = [n * 2 for n in nums]
print(doubled)
```

Exercise 2 · The words in upper case

Write the comprehension that turns the list of words given in the panel into a new list with the words in upper case, then print it exactly as the output below shows. Use this lecture's one-line form, with the string method from week 3.

```
words = ["ada", "grace", "ken"]
# your comprehension here
```

```
>>> ['ADA', 'GRACE', 'KEN']
```

Exercise 3 · Every product a × b

Write the comprehension that builds the list of every product a × b, for a and b from 1 to 3, then print it — exactly as the output below shows. Use this lecture's double comprehension: two for-clauses in one line.

```
>>> [1, 2, 3, 2, 4, 6, 3, 6, 9]
```

Exercise 4 · From a string to a list of ints

Write the comprehension that cuts the string at the commas and makes a list of ints, then print it — exactly as the output below shows. Start from the string given in the panel. Use this lecture's comprehension, with split from week 4 — convert every element as it comes in.

```
text = "10,20,30,40"
# your comprehension here
```

```
>>> [10, 20, 30, 40]
```

Exercise 5 · Predict what it prints

Read the program below and write exactly the list it prints, element by element, in order. On paper, before any run. Then run it and compare — everything in it is taught material, from week 4's split to this lecture's comprehension.

```
text = "1,2;3,4"
rows = [p.split(",") for p in text.split(";")]
print(rows)
```

Exercise 6 · Predict what it prints

Read the program below and write exactly the list it prints. On paper, before any run. Then run it and compare.

```
M = [[1, 2, 3], [40, 50, 60]]
sums = [row[0] + row[1] + row[2] for row in M]
print(sums)
```

Exercise 7 · Every pairing

Write the comprehension that builds, from the two lists given in the panel, the list of every pairing — each word of the first list joined to each word of the second, with a space between. Then print it exactly as the output below shows. Use this lecture's double comprehension: two for-clauses in one line.

```
adjs = ["hot", "cold"]
drinks = ["tea", "coffee"]
# your comprehension here
```

```
>>> ['hot tea', 'hot coffee', 'cold tea', 'cold coffee']
```

Exercise 8 · Predict what it prints

Read the program below and write exactly the list it prints, element by element, in order. On paper, before any run. Then run it and compare.

```
sums = [a + b for a in [10, 20] for b in range(3)]
print(sums)
```

Exercise 9 · The numbers above ten

Write the comprehension that keeps, from the list given in the panel, only the numbers above 10, then print it — exactly as the output below shows. Use this lecture's filter: an `if` after the `for`-clause.

```
nums = [4, 18, 15, 6, 21]
# your comprehension here
```

```
>>> [18, 15, 21]
```

Exercise 10 · Predict what it prints

Read the program below and write exactly the list it prints. On paper, before any run. Then run it and compare.

```
words = ["sun", "moon", "sky", "star"]
short = [w.upper() for w in words if len(w) == 3]
print(short)
```

Recap 1 · Predict what it prints

Read the program below and write exactly what it prints — every line, in order. On paper, before any run. Then run it and compare.

```
words = "ada grace ken"
names = words.split()
sizes = [len(n) for n in names]
total = 0
for n in sizes:
    total = total + n
print(names)
print(sizes, total)
print(n, names[0][1])
```

Recap 2 · Debug the program

The program below should cut the string at the commas, keep the values above 10, and print them with their sum: `[18, 15] 33`. The first run stops with an error; fix it, and it prints the right list with a wrong sum. Find and fix both mistakes — on paper, before any run. Then run it and confirm.

```
text = "4,18,15,6"
nums = [int(n) for n in text.split()]
big = []
total = 0
for n in nums:
    if n > 10:
        big.append(n)
        total = total + 1
print(big, total)
```

Recap 3 · Choose the fragment

`text = "5 10 15 20 25"` is given, and the program ends with `print(picked)`. Choose the fragment that builds picked — every second number, converted to int — so the print shows [5, 15, 25]. One of A to D is right: pick it, and say what each of the others does instead.

```
# A
ns = text.split()
picked = [int(e) for e in ns][::2]
# B
ns = text.split()
picked = [int(e) for e in ns[2:]]
```

```
# C
ns = text.split()
picked = [int(e) for e in ns][1::2]
# D
ns = text.split()
picked = ns[::2]
```

Recap 4 · Predict what it prints

Read the program below and write exactly what it prints — every line, in order. On paper, before any run. Then run it and compare.

```
words = ["go", "run"]
ends = ["!", "?"]
shouts = [w.upper() + e for w in words for e in ends]
total = 0
for s in shouts:
    total = total + len(s)
print(shouts)
print(total, shouts[0])
```

Recap 5 · Debug the program

The program below should cut the string at the semicolons, keep the names longer than three letters in upper case, and print them with their count: `['GRACE', 'HOPPER']` 2. The first run stops with an error — and fixing it is not the end. Remember that a program that runs is not a program that is right. Find and fix every mistake: on paper first, then run and compare.

```
text = "ada;grace;hopper;ken"
names = text.split(",")
long = [n.upper for n in names if len(n) > 3]
count = 0
for n in long:
    count = count + n
print(long, count)
```

Recap 6 · Choose the fragment

`letters = "abc"` is given, and the program ends with `print(pairs)`. Choose the fragment that builds `pairs` — every letter joined to every digit of `"12"`, skipping the letter `"b"` — so the print shows `['a1', 'a2', 'c1', 'c2']`. One of A to D is right: pick it, and say what each of the other three prints instead.

```
# A
pairs = [L + d for L in letters for d in "12" if L != "b"]

# B
pairs = [L + d for d in "12" for L in letters if L != "b"]

# C
pairs = [L + d for L in letters for d in "12" if L == "b"]

# D
pairs = [L for L in letters for d in "12" if L != "b"]
```

NOVA IMS INFORMATION MANAGEMENT SCHOOL

```
input("Questions? ")
```

Acreditações e Certificações da NOVA IMS

eduniversal · QS STARS · ABET Computing Accreditation Commission · iSchools · EMOS · UNIGIS · USGIF · Erasmus Mundus Programme Education and Culture DG · Lapcer · IQNET

Cofinanciado por

PORTUGAL 2030 · 2030 Lisboa · Co-funded by the European Union · fct Fundação para a Ciência e a Tecnologia · PRR Plano de Recuperação e Resiliência · REPÚBLICA PORTUGUESA · Financiado pela União Europeia NextGenerationEU

© Karina Rebuli, 2026

Exercise 11 · Predict what it prints ★

Read the program below and write exactly the two printed lines, in order. On paper, before any run. Then run it and compare.

```
a = [10, 20]
a.append(30)
b = a
b.append(40)
print(a)
print(b)
```